

Odette Project Documentation

Contents

1	Requirements Analysis	1
2	Design, Implementation, Development, Deployment	3
2.1	Design Overview	3
2.2	Implementation	4
2.3	Development	6
2.3.1	Automatic documentation via Doxygen	6
2.3.2	Testing Framework	6
2.4	Deployment	7
3	User Documentation	7
3.1	Getting Started	7
3.2	C++ Core Orbit Propagation Engine	8
3.2.1	Installation Instructions	8
3.2.2	Specification and Usage Instructions	9
3.2.3	Implementation	10
3.3	Python Interface	11
3.3.1	Installation and Setup	11
3.3.2	Core Python Modules	12
3.3.3	Programmatic Python Interface	15
3.3.4	C++ Python Bindings and Design Patterns	15
3.3.5	Comprehensive Logging System	16
3.3.6	Unit Testing Framework	17
3.4	Interactive Command Line Interface (CLI)	17
3.4.1	TLE Operations Menu	17
3.4.2	RK45 Propagation Menu	18
3.4.3	Configuration Management	18
3.4.4	Multi-Satellite Visualization	18
3.4.5	Utilities and Analysis Tools	18
3.5	Java Graphical User Interface (GUI)	19
3.5.1	Installation Instructions	19
3.5.2	Specification and Usage Instructions	21
3.5.3	Implementation	25
3.6	Output Formats and Result Interpretation	27
3.7	Tutorials and Documentation Access	27

1 Requirements Analysis

This section presents an overview of the Odette project from a technical point of view. In the following, a list of core requirements is outlined and discussed, focusing on how they are implemented within the project.

- The project integrates three programming languages, each covering a specific functional domain. A C++ computational core handles the performance-critical orbit propagation and perturbation modeling. A Python layer wraps this core to provide scripting and interactive REPL access. Finally, a Java GUI enables visualization and user-friendly interaction with all the exposed functionality.
- The program supports orbit determination in LEO using both TLE and TDM data. The TLE interface interprets Two-Line Elements to calculate satellite position and velocity using the SGP4 model, stored in json files. (Figure 1)

```
"LAUNCH_DATE": "2016-01-17",
"SITE": "AFWTR",
"DECAY_DATE": null,
"FILE": "4231680",
"GP_ID": "252058821",
"tle_line0": "0 JASON 3",
"tle_line1": "1 41240U 16002A 24078.54273252 -.00000041 00000-0 87212-4 0 9991",
"tle_line2": "2 41240 66.0433 39.6041 0008216 275.2526 84.7552 12.80929447381890"
```

Figure 1: TLE JSON formatting used in the core.

- The C++ codebase can be integrated by including the headers and linking the provided DLLs. The Python wrapper, implemented using pybind11, exposes a simplified interface for REPL or script-based workflows. Java then communicates with Python through a process bridge, enabling GUI integration without directly handling C++.
- The Python wrapper provides a REPL for interactive exploration of orbit propagation capabilities. Users can load TLE or TDM data, configure propagation settings, and view output vectors. (Figure 2)

```
(base) evelynschuller@Evelyns-MacBook-Air Bindings_Python-interface-code % python3 odette_cli.py
=====
ODETTE - Orbit DEtermination TEchnology
Interactive Command Line Interface
=====

MAIN MENU
=====
1. TLE Operations
2. RK45 Propagation
3. Orbit Comparison
4. Multiple Satellites
5. Configuration
6. Utilities
7. Help
8. Exit
=====
Select option: █
```

Figure 2: REPL implemented in Python.

- The Java interface provides a cross-platform GUI that supports data loading, configuration of propagation settings, orbit visualization, and result comparison. The backend is entirely powered by the Python interface. (Figure 3)
- Unit tests are provided for the C++ modules, validating key computations including frame transformations, integration accuracy, and orbital element consistency. (Figure 4)
- The `barlog` namespace implements a lightweight logging framework in C++, which propagates through the Python and Java interfaces. (Figure 5)
- Object-oriented design patterns are applied throughout: Composite and Singleton patterns are used in the Java GUI, while the Python wrapper uses a Facade pattern to abstract the C++ interface. In

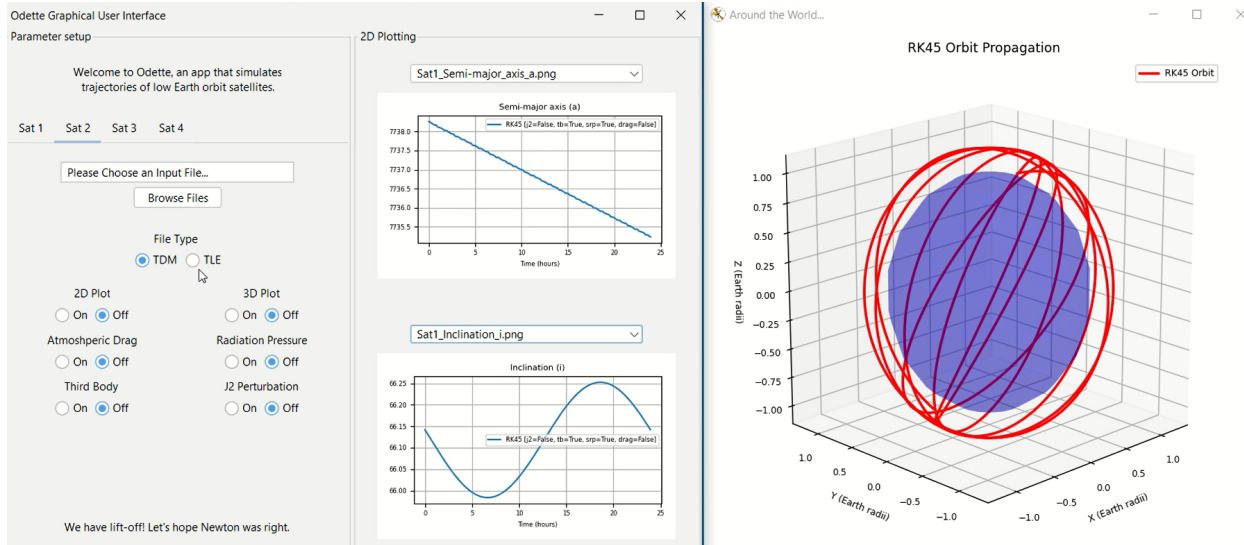


Figure 3: Odette Java GUI.

```
int main() {

    barlog::initialize_log();
    test_radec_solution();
    test_tle_propagation();
    test_rk45_propagation();
    test_orbital_elements_consistency();
    test_perturbation_effects();

    test_angle_conversions();
    test_geodetic_to_ecef();
    test_kepler_acceleration();
    test_constants();

    std::cout << "\n All tests passed successfully.\n";
    return 0;
}
```

Figure 4: C++ unit testing suite.

C++ the Bridge pattern allows the Propagator to dynamically change the integration method through an integrator function.

- GitHub is used for version control, code collaboration, and continuous integration.
- Doxygen is used to generate documentation for the C++, Python, and Java components, with outputs in both HTML and LaTeX formats.
- Tutorial videos demonstrate how to use the application across all three programming interfaces.

2 Design, Implementation, Development, Deployment

2.1 Design Overview

Odette is designed around a modular architecture that clearly separates computational logic from user interface and control flow. The C++ core contains modules for frame transformations, orbit propagators,

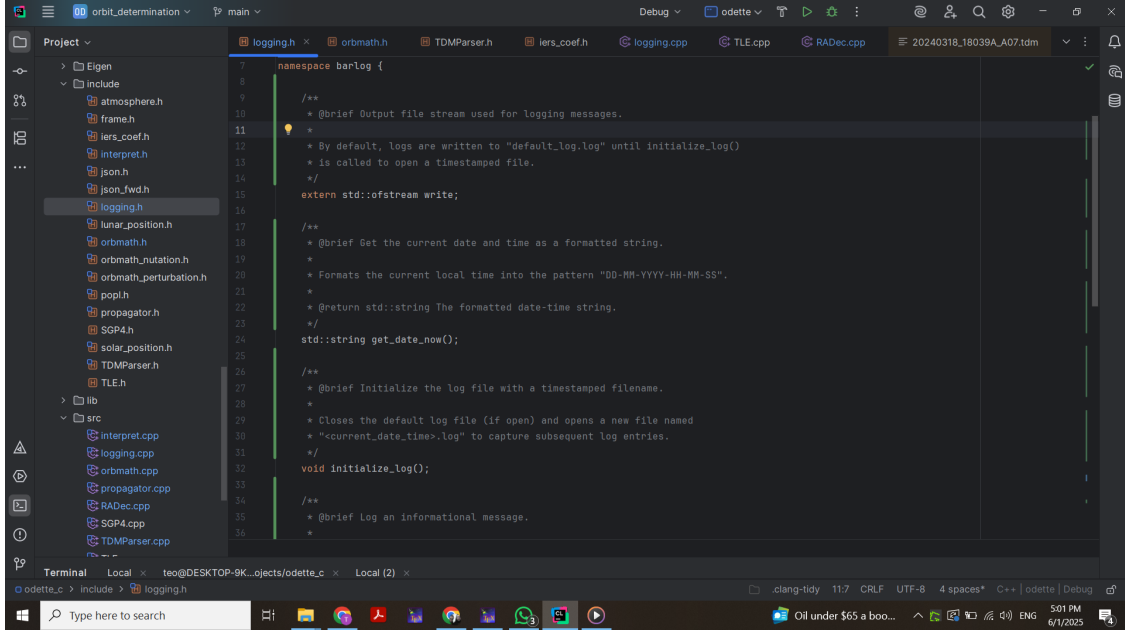


Figure 5: Logging module integrated in all layers.

perturbation models, timekeeping, and data input parsing. All components are loosely coupled and follow consistent naming conventions.

Each module exposes a minimal interface designed for extension and testing. For example, the `orbmath::perturbation` namespace encapsulates J2, atmospheric drag, lunar, and solar pressure effects. The integration layer (`propagator.cpp`) includes RK45 method, with a unified propagation interface.

The Java GUI acts as a view/controller layer, while Python functions as a glue layer, offering both scripting and visualization capabilities. This design promotes platform independence and future expansion, such as the addition of scripting languages or additional visualizations.

2.2 Implementation

The C++ implementation follows clean coding practices and avoids external dependencies where possible. The orbit propagators are written from scratch and implement SGP4 Cowell Method with RK45 integration. The system supports leap second handling, polar motion, and Earth orientation using interpolated IERS parameters.

Multiple libraries were used to properly implement the C++ functionality such as:

CLI Interface

- `popl.h` - Provides command-line interface parsing.
Source: <https://github.com/badaix/popl>

TLE and SGP4 Propagation

- `SGP4.h`, `TLE.h`, `SGP4.cpp`, `TLE.cpp`
Implementation of the SGP4 propagation model using Two-Line Elements.
Source: Adapted from NASA's example at <https://github.com/nasa/ECI/blob/master/examples/sgp4Prop/fsw/src/SGP4.c>

JSON Support

- `json.h`, `json_fwd.h` - JSON parsing and serialization.
Source: <https://github.com/nlohmann/json>

Visualization (for future use)

- `raylib.h`, `raymath.h`, `rlgl.h` - Lightweight 3D graphics and math support.
Source: <https://www.raylib.com>

Solar System Perturbation Models

- `atmosphere.h`, `lunar_position.h`, `solar_position.h`
Implement perturbative models for atmospheric drag, lunar and solar gravity.
Source: https://github.com/Keeby-Astro/Space-Sciences-and-Astroynamics/blob/main/C%20Scripts/Sources/solar_perturbation.cpp

Reference Frame and Earth Orientation

- `frame.h`, `iers_coef.h`
Handle ECEF $\leftrightarrow$ ECI frame transformations and IERS Earth orientation parameters.
Inspired by: <https://github.com/Semi96/ecef2ecij2000/blob/master/ecef2ecij2000.java>

Observation Handling

- `interpret.h`, `TDMParser.h`, `interpret.cpp`, `TDMParser.cpp`, `RADec.cpp`
Modules developed for interpreting RA/Dec and TLE inputs for orbit computation.
Developed

Orbital Mechanics Core

- `orbmath.h`, `orbmath_nutation.h`, `orbmath_perturbation.h`, `orbmath.cpp`
Keplerian motion, nutation, perturbation models, and Earth constants.
Developed

Orbit Propagation

- `propagator.h`, `propagator.cpp`
Encapsulates RK45 and Verlet-based numerical integration of orbits.
Developed

Logging

- `logging.h`
Lightweight stream-based logging utility.
Developed

Python bindings are created using `pybind11`, exposing classes and functions for TLE loading, propagator configuration, and result export. The Python interface consists of three main modules: `orbit_toolkit.py` which provides the `OrbitVisualizer` and `OrbitPropagator` classes for 3D visualization and orbital analysis, `satellite_core_structure.py` which defines core satellite functionality, and `gauss_tle_test.py` for TLE calculation tests using Gauss method. The `JavaGUIBridge` class enables seamless communication between Java and Python through JSON-based command execution. Output vectors and orbital elements can be plotted using `matplotlib` or exported as CSV/JSON files. The Python layer supports comprehensive orbit propagation with configurable perturbation models including J2 Earth oblateness, atmospheric drag, solar radiation pressure, and third-body gravitational effects.

The Java GUI is developed using Java Swing and communicates with Python using JEP (Java Embedded Python) and temporary files. It supports 3D orbit rendering, selection of propagation method, and runtime control of simulation parameters.

2.3 Development

The project uses CMake for cross-platform building and organization. Code is divided across ‘include/’ and ‘src/’ folders for clarity. Extensive testing ensures that core functionality behaves as expected.

The C++ code was compiled using the CMake build system. We expose three different configurations for the project, **odette** which is the main executable configuration, **gauss_tle_test** which provides integration testing and **test_odette** which is the configuration for unit tests.

The development process includes:

- Automatic documentation via Doxygen.
- Testing framework for C++ and interactive Python validation.
- Continuous versioning and releases on GitHub.
- Visual documentation through videos and GUI demos.

2.3.1 Automatic documentation via Doxygen

We employed the tool ”Doxygen” to generate documentation in both HTML and LaTeX formats using the code’s comments, which were formatted according to the tool’s specification, allowing us to automatically update the information after every new feature. (Figure 6)

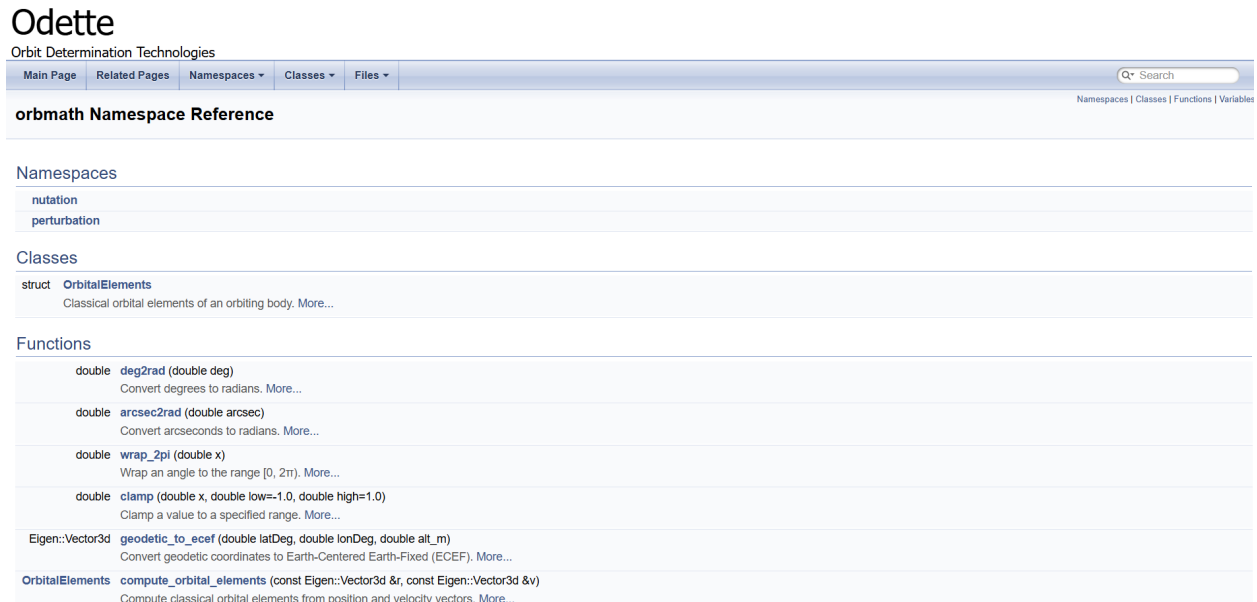


Figure 6: HTML of the C++ part.

The Python interface documentation is also generated using Doxygen and includes comprehensive API reference documentation covering the **OrbitVisualizer**, **OrbitPropagator**, and **JavaGUIBridge** classes. The documentation provides detailed method signatures, parameter descriptions, return values, and usage examples for all public interfaces. The HTML documentation includes cross-references between related classes and methods, making it easy to navigate the Python API structure.

2.3.2 Testing Framework

Comprehensive testing was implemented for each namespace in order to verify the well functioning and correctness of the implementation. No testing framework was used but instead, we created a test-specific configuration in the CMakeLists.txt file. (Figure 7)

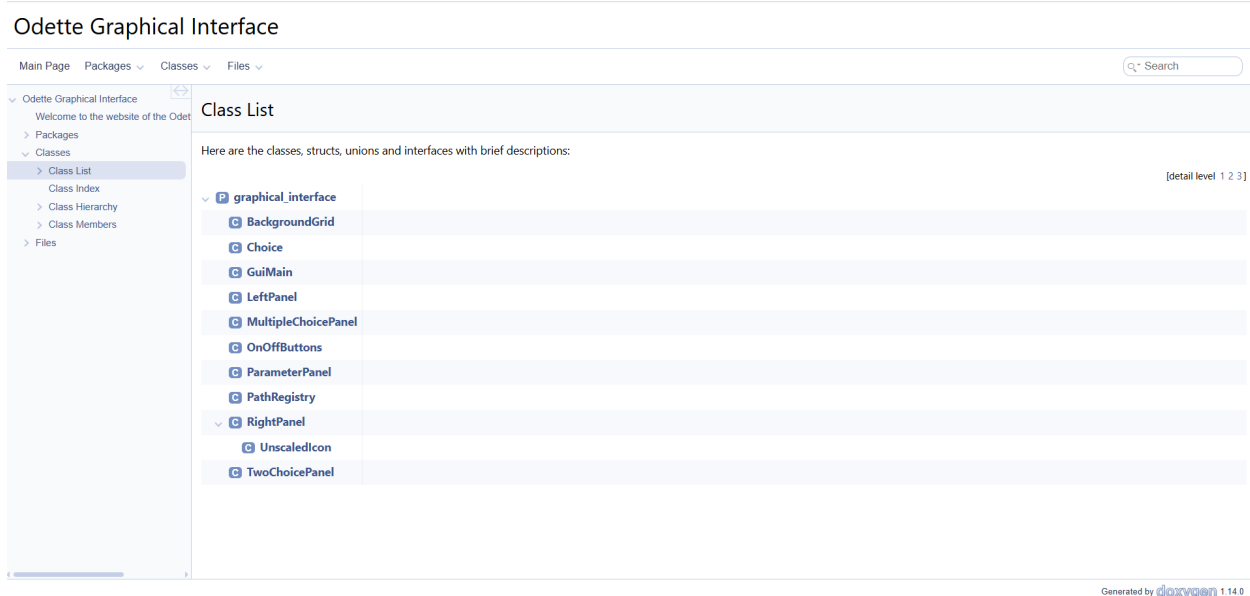


Figure 7: Webpage of the Java part.

The Python interface includes several test modules: `gauss_tle_test.py` validates TLE calculations using the Gauss method, while the `gauss_tle_test_results/` directory stores comprehensive test outputs. The testing framework verifies orbital element calculations, propagation accuracy between SGP4 and RK45 methods, and proper handling of various perturbation models. Interactive Python validation is performed through the REPL interface, allowing real-time testing of orbit visualization and propagation functions.

2.4 Deployment

Deployment involves building the C++ core as a shared library, setting up the Python virtual environment, and running the Java GUI through the JAR file.

The Python layer can be used independently in Jupyter notebooks or scientific workflows. The Java GUI is cross-platform and includes built-in sample files.

3 User Documentation

The Odette orbit determination system supports three main modes of user interaction: a command-line interface (REPL) using Python, a cross-platform graphical user interface (GUI) built in Java, and a programmatic Python interface for scripting and scientific analysis. Each of these interfaces provides access to core functionality exposed by the C++ engine, including orbit propagation, perturbation modeling, frame transformations, and input/output management.

This section presents a comprehensive guide to using Odette from each interface, including example workflows, usage recommendations, and customization tips.

3.1 Getting Started

Before using any interface, users must ensure that the Odette system is correctly installed. This typically involves the following steps:

- Building the C++ core using CMake. The build process outputs a shared library (DLL or `‘.so’/‘.dylib’`) which is used by the Python and Java layers.

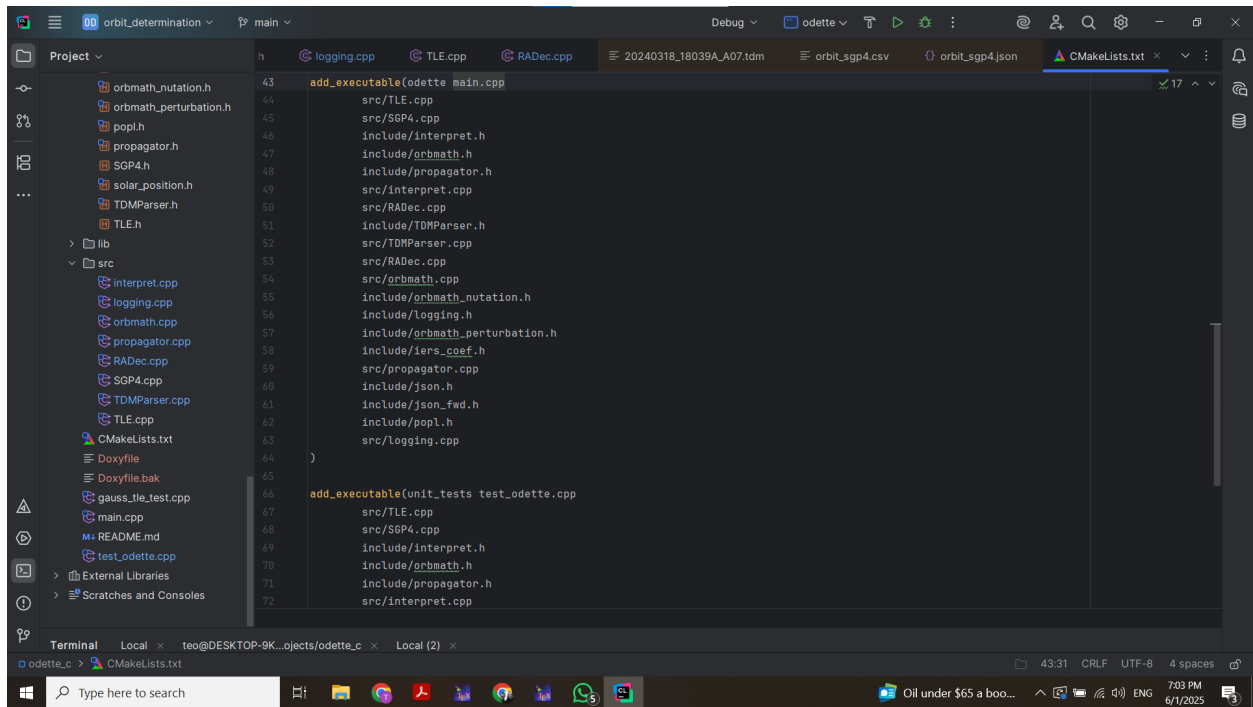


Figure 8: CMake list.

- Setting up a Python virtual environment and installing dependencies via ‘pip’, including ‘pybind11’, ‘matplotlib’, and ‘numpy’.
- Running the provided python script or launching the Java JAR file to start the GUI.

3.2 C++ Core Orbit Propagation Engine

3.2.1 Installation Instructions

Note: The C++ backend is designed for UNIX-like systems and requires a GCC-compatible compiler. It does not compile with MSVC on Windows. Users are advised to install WSL or use a Linux distribution for development.

To build the C++ engine, follow these steps:

1. Navigate into the **build** directory:

```
cd build
```

2. Generate Makefiles with CMake:

```
cmake ..
```

3. Copy the required shared libraries into the **build** folder:

- On Windows, copy the **.dll** files from **lib**
- On Linux, copy the **.so** files from **lib**

4. Move the simulation input files into **build**:


```
mv constants.csv build/  
mv data build/
```

5. Compile the application:

```
make
```

After compilation, the executables (e.g., `odette`, `gauss_tle_test`) will appear in the `build` directory. These can be used directly or invoked via Python/Java bindings.

3.2.2 Specification and Usage Instructions

The Odette C++ core handles all of the low-level computations involved in orbit determination and propagation. It is designed for performance and modularity, exposing key functionality through well-organized namespaces and function interfaces. The computational code operates independently of the graphical interface or Python REPL and can be compiled and executed on its own or integrated via dynamic libraries.

There are three main executable programs associated with the C++ implementation:

- `./odette` – The main orbit simulation executable. This reads input paths (hardcoded in source) and performs propagation for a specified satellite using RK45 or SGP4 methods.
- `./gauss_tle_test` – A testing executable that reads TLE data from a JSON file and applies SGP4 method to estimate initial states. Results are propagated and printed for verification. Similarly, TDM files are read and Gauss' method is applied for generating the position and velocity vectors.
- `./test_odette` – Runs a suite of regression and validation tests for propagation accuracy, file parsing, frame conversions, and orbital element extraction.

The core supports two types of input formats:

- **TDM** (Tracking Data Message) files – The user must provide at least three files per satellite, inside a directory. These files include timestamps, observed angles, and optionally range data.
- **TLE** (Two-Line Element) files – The data is expected in JSON format, with entries for both TLE lines, satellite name, and epoch.

To run an orbit simulation:

- Place your TLE JSON file or TDM folder in the `/data` directory.
- Use `./gauss_tle_test` or `./odette` depending on the input format.
- Review the output in the console or in generated CSV/JSON files located in the `/output` directory.

The system supports enabling/disabling perturbations, and results include position and velocity vectors over time, along with derived orbital elements. Python and Java interfaces use this exact backend under the hood, meaning all GUI and REPL simulations go through these exact steps, abstracted from the user.

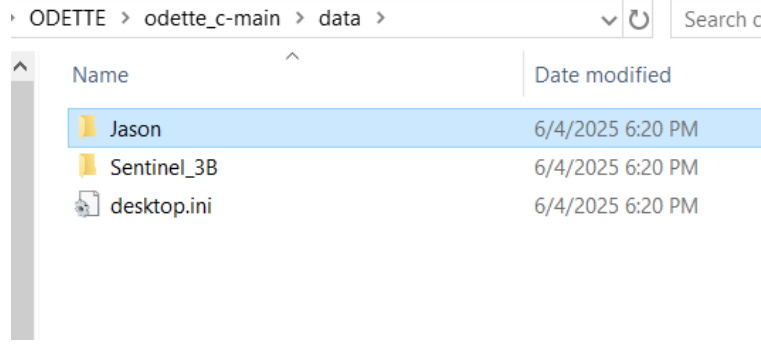


Figure 9: Structure of the C++ orbit folder. Input data should be placed in the **data** folder.

3.2.3 Implementation

The C++ code for the Odette core consists of multiple namespaces, each grouping a specific part of the computational logic. Each namespace is composed of header and source files, most of which are compiled into a shared library that is later used by Python and Java. Below, we provide an overview of the most relevant namespaces and classes, together with short explanations regarding their functionality.

- **barlog**

A lightweight logging system that wraps output to the terminal with severity tags. Used for debugging and monitoring execution.

Functions:

- `barlog::info(msg)` – Outputs info-level log.
- `barlog::warn(msg)` – Outputs warning.
- `barlog::error(msg)` – Outputs error messages with traceback context.

- **interpret**

This namespace handles input parsing and initial orbit estimation. It can read TDM folders or JSON-encoded TLE files and apply classical methods for orbit determination.

Functions:

- `parse_tdm(folder_path)` – Reads RA/Dec observations from multiple TDM files.
- `gauss_solve(observations)` – Applies Gauss’s method to compute initial state vectors from three observations.
- `parse_tle_json(file)` – Reads TLE from JSON and converts to internal format.

- **orbmath**

Contains core mathematical logic, including vector/matrix operations, Keplerian mechanics, and coordinate transformations.

Functions:

- `compute_orbital_elements(r, v)` – Converts a position/velocity vector to Keplerian elements.
- `eci_to_ecef(r_eci, time)` – Frame transformation for Earth orientation.
- `cross(a, b)` – Computes the cross product of two vectors.

- **orbmath::perturbation**

Perturbation models applied to orbit propagation. All models return acceleration vectors that are summed with the central force.

Models available:

- J2 (Earth oblateness)
- Atmospheric drag
- Solar radiation pressure
- Third body gravity (Sun, Moon)

Each perturbation has its own toggle flag that can be enabled before propagation.

- **propagate**

The core simulation engine. Contains numerical integrators and the `Propagator` class.

Classes:

Propagator

- Stores initial state vector and simulation parameters.
- Supports RK45 and Verlet integration methods.
- Applies perturbation models conditionally.
- Generates `Ephemeris` object with time series of state vectors.

Ephemeris

- Holds time-aligned records of position and velocity.
- Supports CSV/JSON export, orbit element computation, and visualization hooks.

Functions:

- `rk45_eci(init_state, t0, t1, step, perturb)` – Runge-Kutta 4(5) propagation.
- `verlet(init_state, t0, t1, step, perturb)` – Velocity-Verlet integration.

The C++ implementation is thoroughly tested through the `test_odette` suite. It ensures consistency between reference orbits and internally propagated ones, handles invalid file formats gracefully, and validates the accuracy of orbital element extraction. The system is compiled using CMake and has minimal dependencies, allowing portability across UNIX-like systems and integration into Python via pybind11.

Each of the above namespaces is reflected in the Python bindings and GUI interaction logic. This modularity allows developers to test or extend each part independently and reuse the core logic across both command-line and graphical interfaces.

3.3 Python Interface

The Python interface provides comprehensive access to the Odette orbital mechanics capabilities through three main approaches: an interactive REPL for command-line exploration, a programmatic API for scripting and scientific analysis, and a bridge interface for Java GUI integration. The Python layer is built using pybind11 bindings that expose the high-performance C++ computational core while providing a Pythonic interface for ease of use.

3.3.1 Installation and Setup

The Python interface requires the following dependencies:

- Python 3.8 or later
- pybind11 for C++ bindings
- matplotlib for plotting and visualization
- numpy for numerical operations

Installation on Windows requires Microsoft Visual C++ Build Tools. The build process is handled through `setup.py` which compiles the C++ extensions and creates the necessary Python modules. After successful installation, the `satellite_core` extension module (with `.pyd` extension on Windows) provides the Python-C++ interface. The Odette python interface can be installed as a standalone application by downloading the `python-interface.zip` archive from GitHub: https://github.com/teomatei22/odette_c/tree/python-interface.

3.3.2 Core Python Modules

The Python interface consists of several key modules:

orbit_toolkit.py This is the main module providing three primary classes:

OrbitVisualizer Class

The `OrbitVisualizer` class provides comprehensive 3D visualization capabilities for satellite orbits, creating matplotlib-based plots with Earth representation.

Key Methods:

- `visualize_tle(tle_line1, tle_line2, time_span_hours=6, step_minutes=0.5)` – Visualizes satellite orbit using TLE data with SGP4/SDP4 propagation. Converts positions to Earth-Centered Inertial coordinates and normalizes to Earth radii.
- `visualize_rk45(tdm_wildcard, delta, delta_error, perturbations=None)` – Visualizes orbit using RK45 numerical integration, supporting various perturbation models including J2 Earth oblateness, third-body effects, solar radiation pressure, and atmospheric drag.
- `compare_orbits(tle_line1, tle_line2, tdm_wildcard, perturbations_list=None)` – Compares TLE and RK45 orbit propagation methods side-by-side on the same 3D plot.
- `visualize_multiple_satellites(satellites_data)` – Visualizes multiple satellites simultaneously with different colors and labels.

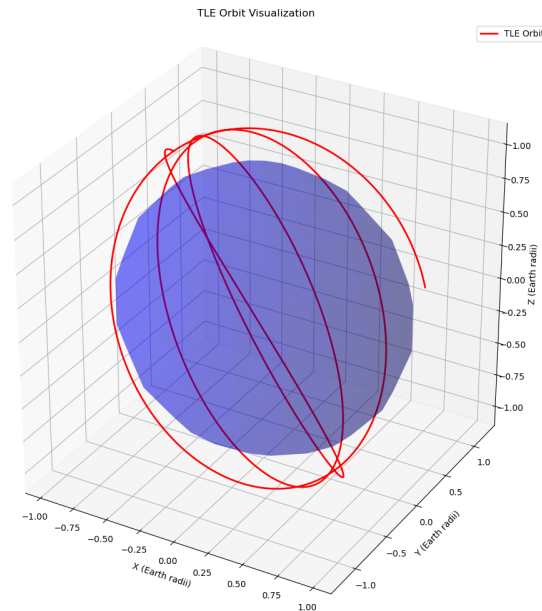


Figure 10: TLE orbit visualization using SGP4 propagation method, using `visualize_tle()` method. The 3D trajectory shows the satellite's orbital path over a specified time period, with Earth rendered as a blue sphere for reference. Position data is computed using the SGP4/SDP4 analytical propagation model from Two-Line Element data.

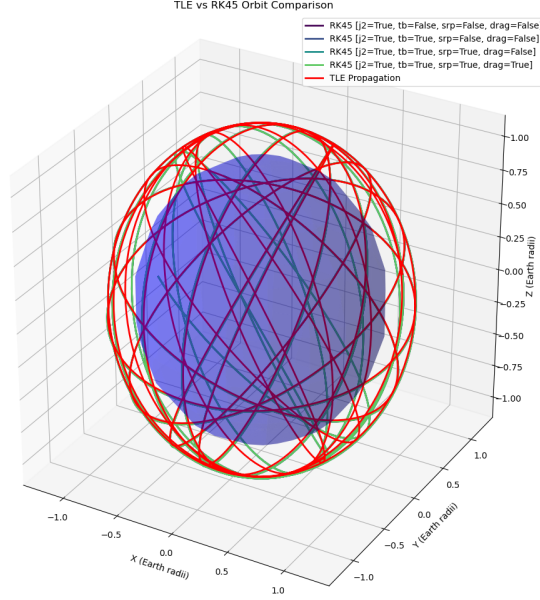


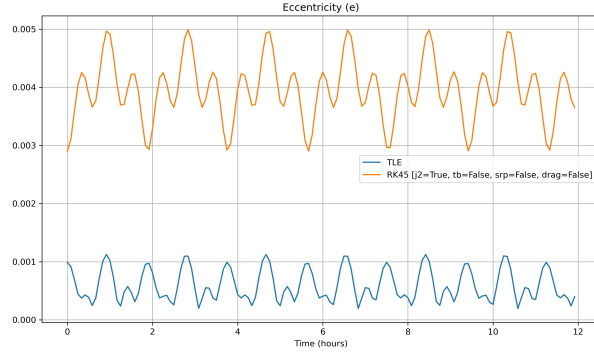
Figure 11: Comparison of TLE (SGP4) versus multiple RK45 numerical integration methods, using `visualize_multiple_satellites()` method. The visualization displays different orbital trajectories computed using SGP4 analytical propagation compared with RK45 numerical integration under various perturbation configurations, demonstrating the differences between analytical and numerical propagation approaches.

OrbitPropagator Class

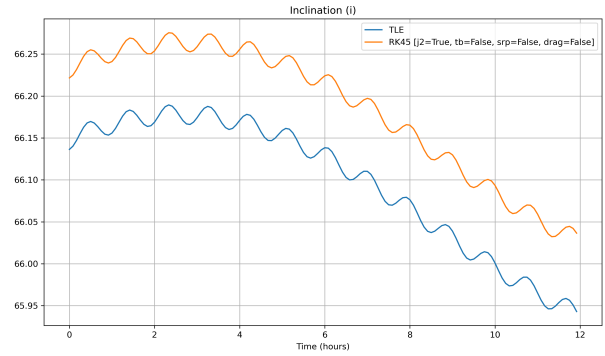
The `OrbitPropagator` class extends `OrbitVisualizer` to provide advanced orbit propagation with detailed orbital elements analysis.

Key Methods:

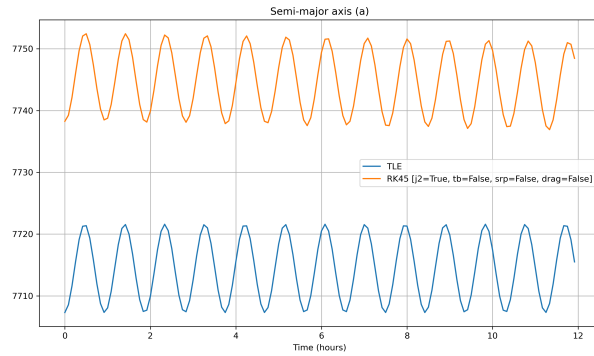
- `propagate_tle(tle_line1, tle_line2, time_span_hours=6)` – Performs TLE-based orbit propagation returning detailed orbital elements including semi-major axis, eccentricity, inclination, RAAN, argument of perigee, true anomaly, and mean anomaly.
- `propagate_rk45(tdm_wildcard, delta, delta_error, perturbations=None)` – Performs high-precision numerical orbit propagation using RK45 integration with configurable perturbation models.
- `compare_orbital_elements()` – Comprehensive method for comparing orbital trajectories between different propagation methods, generating statistical comparisons and difference plots.
- `set_perturbation_parameters(params)` – Configures physical parameters for perturbation force modeling including solar radiation pressure coefficient, area-to-mass ratio, and drag coefficient.
- `get_perturbation_parameters()` – Retrieves current values of all perturbation model parameters.



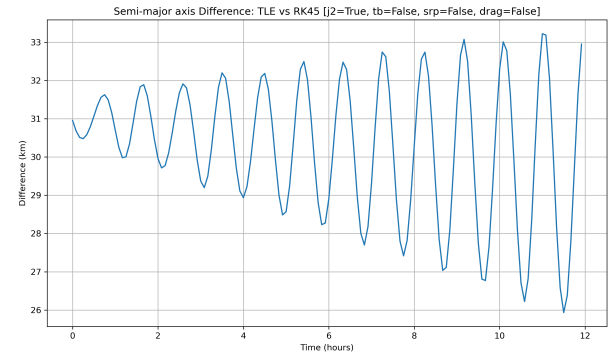
(a) Orbital eccentricity evolution over time using RK45 numerical integration. The plot shows how the orbital eccentricity parameter varies as a function of time when accounting for perturbative forces through high-precision RK45 numerical propagation methods.



(b) Orbital inclination evolution using RK45 propagation method. The time series demonstrates the variation of orbital inclination due to perturbative effects computed through RK45 numerical integration.



(c) Semi-major axis evolution computed using RK45 numerical propagation. The plot illustrates how the orbital semi-major axis changes over time when perturbative forces are included in the RK45 numerical integration scheme.



(d) Comparison between TLE (SGP4) and RK45 methods with J2 perturbation only. The orbital element evolution shows differences between SGP4 analytical propagation and RK45 numerical integration with Earth's oblateness.

Figure 12: Orbital element evolution and propagation method comparison, using `compare_orbital_elements()` method. The panels show (a) eccentricity, (b) inclination, (c) semi-major axis evolution using RK45 method, and (d) TLE vs RK45 comparison with J2 perturbation enabled.

JavaGUIBridge Class

The `JavaGUIBridge` class serves as a standardized interface for Java applications to interact with the Python orbital mechanics code through JSON-based command execution.

Key Features:

- JSON command parsing and execution for seamless Java-Python communication
- Support for all visualization and propagation commands with standardized response formats
- Progress callback mechanism for status updates during long-running operations
- Comprehensive error handling with structured error responses
- Data conversion utilities for NumPy arrays and scientific data structures

satellite_core.structure.py Defines core satellite data structures and functionality, providing object-oriented representations of satellite states and orbital parameters.

gauss_tle_test.py Implements TLE calculation tests using the Gauss method for orbit determination. This module validates the accuracy of initial orbit determination from observational data and compares results between different propagation methods.

3.3.3 Programmatic Python Interface

Advanced users can import the Odette Python module directly in scripts for automated analysis and integration into larger scientific workflows:

```
from odette import OrbitVisualizer, OrbitPropagator

# Create TLE propagator
propagator = OrbitPropagator()
results = propagator.propagate_tle(line1, line2, time_span_hours=24)

# Analyze orbital elements
initial_sma = results['orbital_elements'][0]['a']
final_sma = results['orbital_elements'][-1]['a']
print(f"SMA change: {final_sma - initial_sma:.3f} km")

# Visualize multiple satellites
visualizer = OrbitVisualizer()
satellites_data = [
    {'type': 'tle', 'tle_line1': line1, 'tle_line2': line2,
     'color': 'red', 'label': 'Satellite 1'},
    {'type': 'rk45', 'tdm_wildcard': 'data/*.tdm',
     'perturbations': {'j2': True, 'atmospheric_drag': True},
     'color': 'blue', 'label': 'Satellite 2'}
]
visualizer.visualize_multiple_satellites(satellites_data)
```

This programmatic interface enables users to:

- Create and manipulate propagation configurations with custom parameters
- Access raw ephemeris data as structured dictionaries and arrays
- Run simulations with varying perturbation models for sensitivity analysis
- Generate custom plots and visualizations using matplotlib integration
- Automate orbital analysis across multiple satellites or time periods
- Export results in various formats (CSV, JSON) for further analysis

3.3.4 C++ Python Bindings and Design Patterns

The Python interface utilizes pybind11 to create high-performance bindings to the C++ computational core. The bindings are implemented in **satellite_core_bindings.cpp** and expose a comprehensive set of C++ functionality through the **satellite_core** Python module.

Exposed C++ Components:

- **TLE and SGP4 Classes:** Direct binding of Two-Line Element parsing and SGP4 propagation functionality, including position/velocity computation and epoch handling
- **Propagation Module** (**satellite_core.propagate**): Advanced orbit propagation including the Propagator class, Ephemeris containers, and numerical integrators (RK45, Verlet)

- **Frame Transformations** (`satellite_core.frames`): ECEF $\leftrightarrow$ ECI coordinate conversions with IERS corrections and astronomical nutation models
- **Orbital Mathematics** (`satellite_core.orbmeth`): Keplerian mechanics, orbital element computation, and coordinate transformations
- **Perturbation Models** (`satellite_core.orbmeth.perturbation`): J2 Earth oblateness, atmospheric drag, solar radiation pressure, and third-body gravitational effects
- **Nutation Models** (`satellite_core.orbmeth.nutation`): IAU-1980 nutation, precession matrices, and polar motion corrections
- **Data Interpretation** (`satellite_core`): TDM parsing, observation handling, and initial orbit determination using Gauss's method

Design Patterns in Python Interface:

The `orbit_toolkit_module.py` implements several key design patterns:

- **Factory Pattern:** The `create_orbit_logger()` function in `orbit_toolkit_logging.py` serves as a factory for creating properly configured `OrbitLogger` instances with component-specific settings and multiple specialized log streams
- **Decorator Pattern:** The `@performance_monitor` and `@visualization_monitor` decorators provide automatic performance tracking, error handling, and logging capabilities that can be applied to any function without modifying its core logic
- **Facade Pattern:** The `OrbitVisualizer` and `OrbitPropagator` classes serve as simplified facades that hide the complexity of the underlying C++ `satellite_core` module while providing intuitive Python interfaces
- **Bridge Pattern:** The `JavaGUIBridge` class acts as a bridge between the Java GUI and Python computational components, providing JSON-based command execution and standardized data serialization

3.3.5 Comprehensive Logging System

The `orbit_toolkit_logging.py` module provides a sophisticated logging framework with multiple specialized log streams:

- **General Application Log:** Standard INFO-level logging for application events with both file and console output
- **Structured JSON Log:** Machine-readable JSON Lines format for data analysis and automated processing
- **Error-Specific Log:** Dedicated ERROR-level logging with full exception tracebacks and contextual information
- **Performance Metrics Log:** Timing and resource utilization tracking for optimization and bottleneck identification

The logging system includes decorators for automatic event tracking:

- `@performance_monitor`: Automatically captures execution time, success/failure status, and performance metrics
- `@visualization_monitor`: Specialized monitoring for visualization operations with rendering performance tracking

3.3.6 Unit Testing Framework

The `test_orbit_toolkit.py` module provides comprehensive unit testing coverage including:

- **Component Tests:** Individual testing of `OrbitPropagator`, `OrbitVisualizer`, and `JavaGUIBridge` classes
- **Integration Tests:** End-to-end workflow testing ensuring proper component interaction and data flow
- **Edge Case Testing:** Boundary condition testing including zero time spans, large datasets, and invalid input handling
- **Performance Tests:** Memory efficiency testing with large datasets and concurrent operation validation
- **Logging Integration Tests:** Verification of proper logging integration across all components

The test suite utilizes extensive mocking to isolate components and validate behavior without requiring actual satellite data or complex C++ dependencies. Tests include detailed documentation and provide comprehensive coverage reports for quality assurance.

3.4 Interactive Command Line Interface (CLI)

The Python CLI provides a comprehensive text-based interface implemented in `odette_cli.py`, offering full access to ODETTE's capabilities through an intuitive menu-driven system. The CLI employs the **Command Pattern** design, where user actions are encapsulated as command objects that can be executed, queued, and undone.

When launched, the CLI presents a structured main menu:

```
=====
                ODETTE - Orbit DETermination TEchnology
                Interactive Command Line Interface
=====

=====
MAIN MENU
=====
1.  TLE Operations
2.  RK45 Propagation
3.  Orbit Comparison
4.  Multiple Satellites
5.  Configuration
6.  Utilities
7.  Help
8.  Exit
-----
```

3.4.1 TLE Operations Menu

The TLE operations menu provides comprehensive Two-Line Element functionality:

- **Visualize TLE orbit:** Creates 3D orbit visualizations with configurable time spans and step sizes
- **Propagate TLE orbit:** Performs SGP4 propagation returning detailed orbital elements and position/velocity vectors
- **Load TLE from file:** Automatic TLE parsing from standard format files with validation
- **Enter TLE manually:** Interactive TLE input with real-time format validation

3.4.2 RK45 Propagation Menu

Advanced numerical integration capabilities include:

- **Visualize RK45 orbit:** High-precision trajectory visualization with perturbation modeling
- **Propagate RK45 orbit:** Numerical orbit propagation with configurable integration parameters
- **Configure perturbations:** Interactive setup of J2, third-body, solar radiation pressure, and atmospheric drag models
- **Set perturbation parameters:** Physical parameter configuration (area-to-mass ratio, drag coefficients, etc.)

3.4.3 Configuration Management

The CLI implements persistent configuration management allowing users to:

- Set default time parameters (time spans, step sizes)
- Configure default perturbation settings
- Manage TDM parsing parameters (delta, delta_error)
- Control visualization settings (plot display, save paths)
- Save/load configuration profiles to JSON files
- Reset to factory defaults

3.4.4 Multi-Satellite Visualization

The CLI supports complex multi-satellite scenarios with:

- Mixed TLE and RK45 satellite configurations
- Individual color and label management
- Dynamic satellite addition/removal
- Comparative trajectory analysis

3.4.5 Utilities and Analysis Tools

Advanced utility functions include:

- TLE to orbital elements conversion with full Keplerian parameter extraction
- Orbital period calculation using Kepler's third law
- TLE format validation with detailed error reporting
- TDM file management and discovery
- Result export in CSV and JSON formats
- Perturbation parameter inspection and modification

Use Case: Complete Orbit Analysis Workflow

1. Launch the CLI: `python odette_cli.py`
2. Navigate to **TLE Operations** → **Load TLE from file**
3. Select **Propagate TLE orbit** with custom time span (e.g., 24 hours)
4. Switch to **RK45 Propagation** → **Configure perturbations**
5. Enable desired perturbation models (J2, atmospheric drag)
6. Use **Orbit Comparison** → **Compare TLE vs RK45**
7. Generate statistical analysis and visualization plots
8. Export results via **Utilities** → **Export results to CSV**

The CLI automatically handles error conditions, provides helpful defaults, and offers context-sensitive help throughout the user experience.

3.5 Java Graphical User Interface (GUI)

3.5.1 Installation Instructions

Note: In their current form, the Java GUI files ensure Windows functionality. Users with other operating systems might encounter some compatibility issues, especially coming from the Python-C++ binding.

The Odette graphical interface can be installed as a standalone application by downloading the **Java-Interface.zip** archive from GitHub: https://github.com/teomatei22/odette_c/tree/Java-Interface. It is not necessary to download the C++ and Python files separately, as they are already included in the archive.

After unzipping the archive, the user obtains two folders: **odette_c-python-interface**, which contains the C++ and Python files, and **JavaInterface**, which contains the GUI files. The GUI source files are found under **JavaInterface\JavaProject\src\graphical_interface**, with **GuiMain.java** begin the driver class of this application. Inside the **JavaInterface** folder we also notice the **Data** folder, which contains example input files for orbit simulation, as well as the **Plots** folder, which stores the plots generated in the simulation.

In order to get the application running, there are 3 **.jar** files that need to be included in the Java project created by the user (Important: the Java project should be created in the **JavaProject** folder, using the files provided). These files are found in the **JavaInterface** folder: **flatlaf-3.6.jar**, **jep-4.2.2.jar**, **json-20250517.jar**. They allow us to import libraries responsible for changing the Look and Feel of the interface for a more modern design, to create the Java-Python binding, and respectively to work with JSON objects. For those using Eclipse as their IDE, the process is quite simple. For other IDEs, specific instructions might be found online.

In Eclipse, we begin by creating a new project based on the provided source files. Next, we right-click on the project name in the left-side bar, and then go to Build Path → Configure Build Path → Java Build Path → Libraries. We click on Classpath, and then on Add External JARs, to add the JAR files individually. For the JEP JAR file, there is one additional step: we expand the JEP line, and go to Native Library Location, and then click edit. Here, we select the folder **jep** from the **JavaInterface** parent folder contained in the downloaded archive. With these settings in place, the graphical interface can be started by running the **GuiMain.java** file from the IDE.

Name	Date	Type	Size
Data	23-May-25 17:49	File folder	
JavaProject	23-May-25 14:44	File folder	
jep	23-May-25 15:51	File folder	
Plots	23-May-25 16:22	File folder	
flatlaf-3.6.jar	15-Apr-25 18:52	Executable Jar File	904 KB
jep-4.2.2.jar	10-May-25 9:31	Executable Jar File	165 KB
json-20250517.jar	24-May-25 12:45	Executable Jar File	81 KB

Figure 13: Structure of the JavaInterface folder. In the user's IDE, the project files should be placed inside the JavaProject folder.

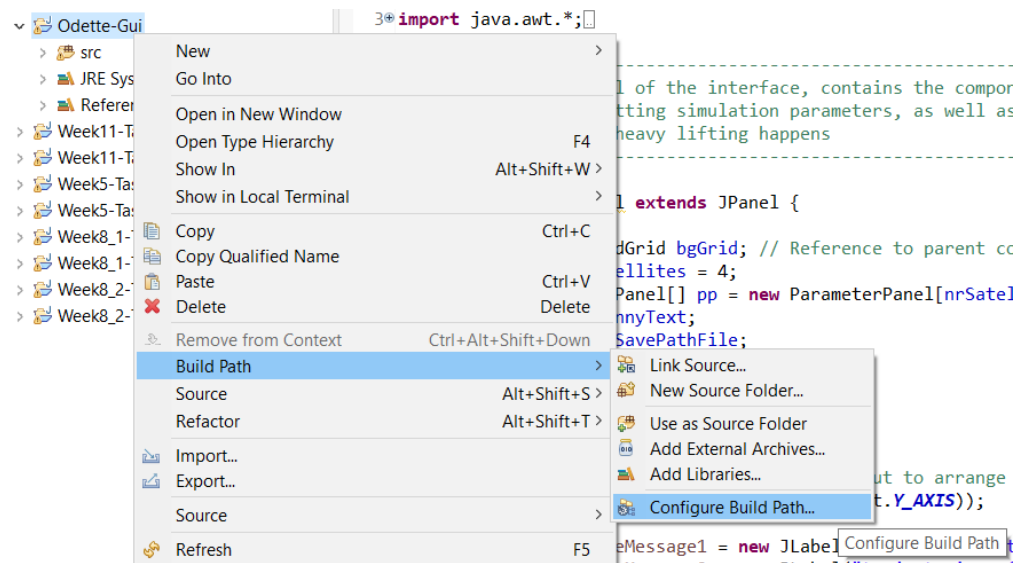


Figure 14: Configuring the project build path in Eclipse.

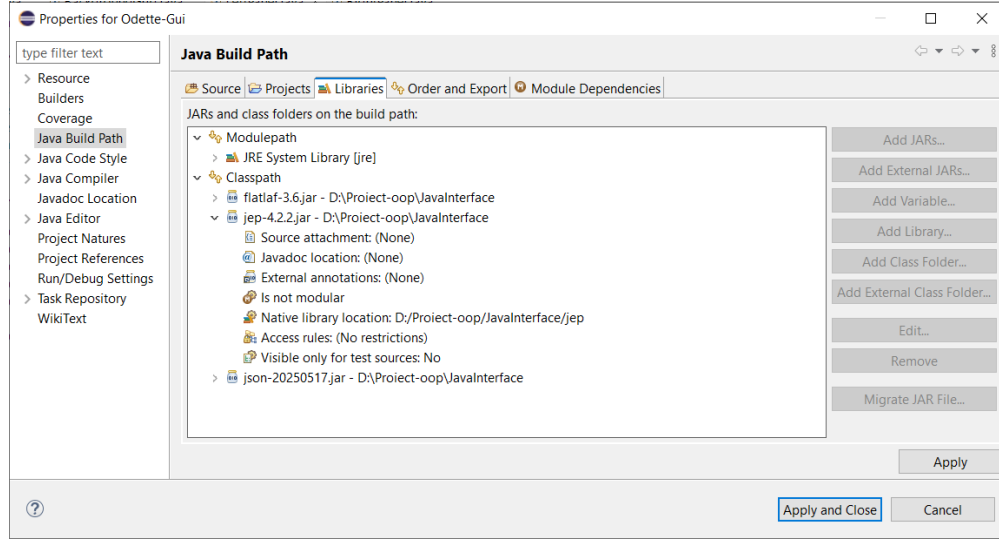


Figure 15: Adding external JAR files in Eclipse.

3.5.2 Specification and Usage Instructions

The Odette graphical interface allows the user to access the computational capacity of our C++ and Python software without requiring any programming knowledge. With a clean and simple layout, the interface allows the simulation of up to 4 satellites at the same time, requiring minimal effort.

The interface is made up of three main areas: upon launching, the user is met with two windows: the left window is meant for setting up simulation parameters, such as input files, desired plot types, and orbit perturbations; the right window displays plots of orbital elements corresponding to each individual simulated satellite. After running the simulation, the right window is responsible with displaying the generated 2 dimensional plots, which can be selected from the two ComboBoxes present on the right panel. Again, after running the simulation, the third window is opened next to the right window. This will contain a 3 dimensional representation of the satellite orbit, which can be rotated with the mouse in order to inspect the trajectory from various angles.

The steps to perform an orbit simulation are as follows:

- Select satellite to simulate from the list Sat1 - Sat4.
- For a given satellite, select desired input files in TDM or TLE format using the Browse Files button.
Please Note: when simulating orbits with TDM input files, the program has to be provided with at least 3 TDM files associated to the satellite's motion, and these files have to be placed inside a folder - the folder is then chosen as input file for the satellite. For TLE input, the data has to be placed inside a JSON file. Consult the example files in the Data folder for more details.
- Select type of input file for a given satellite.
- Select desired plot types for a given satellite: 2D or 3D plotting.
- Select desired perturbations for a given satellite. The list of possible perturbations is comprised of: air drag, solar radiation pressure, third body perturbation, and J2 perturbation.
- After choosing simulation parameters for up to 4 satellites, press the Take Off button.

After a few seconds, the user should notice the plots that appear in the right window (now the central window, after the appearance of the 3D plot). Next to it, we have the 3 dimensional plot of the satellite orbit.

Due to the nature of the underlying computational code, we can only generate a 3D representation for up to 1 TDM file and 1 TLE file at the same time. There are no restrictions for the 2D plots, however. If

the user requests the generation of a 3D plot for more than 1 TDM and 1 TLE input files at the same time, they will be met with a pop-up message on the screen, explaining the maximum accepted number of input files.

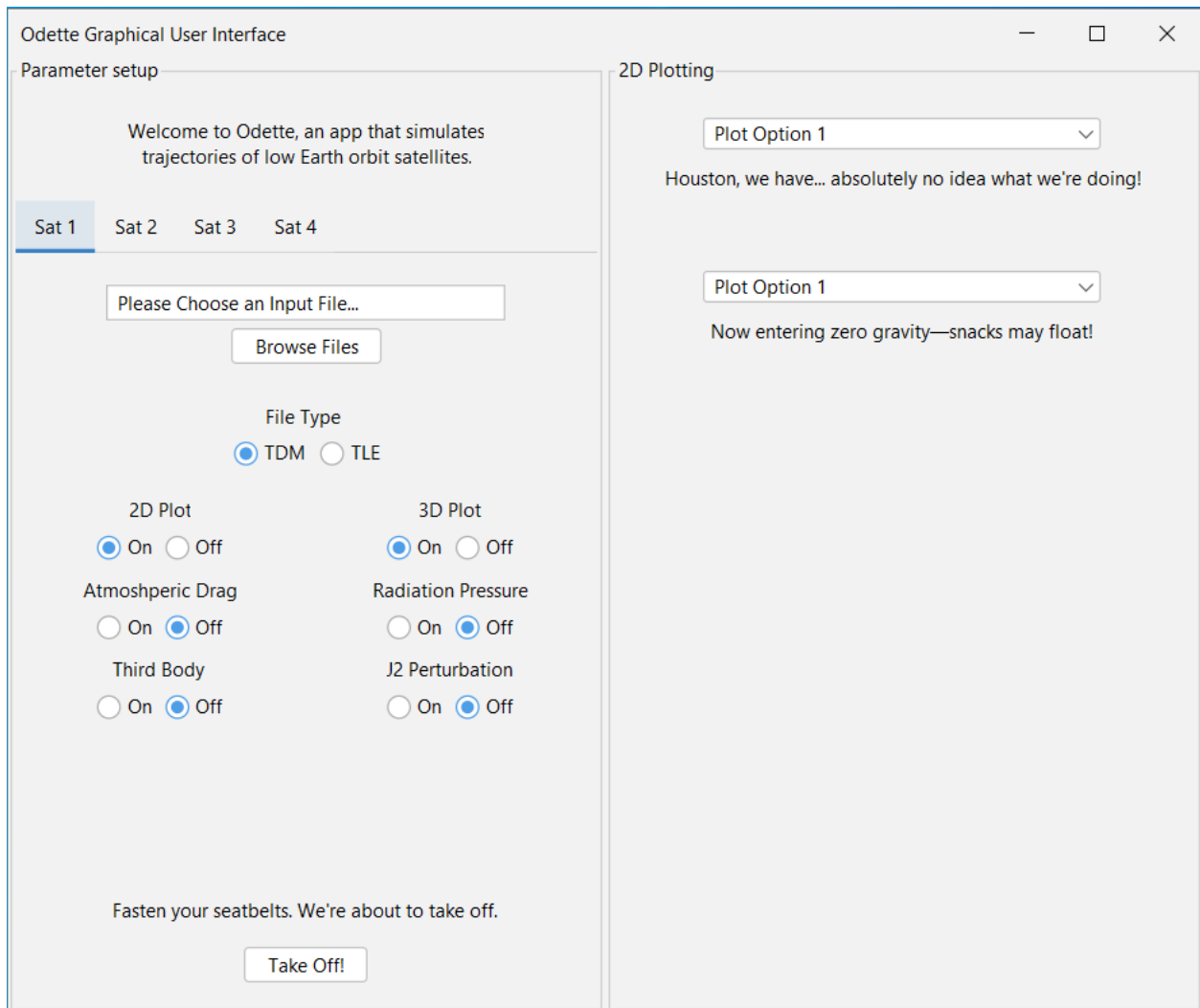


Figure 16: The Odette graphical interface upon opening.

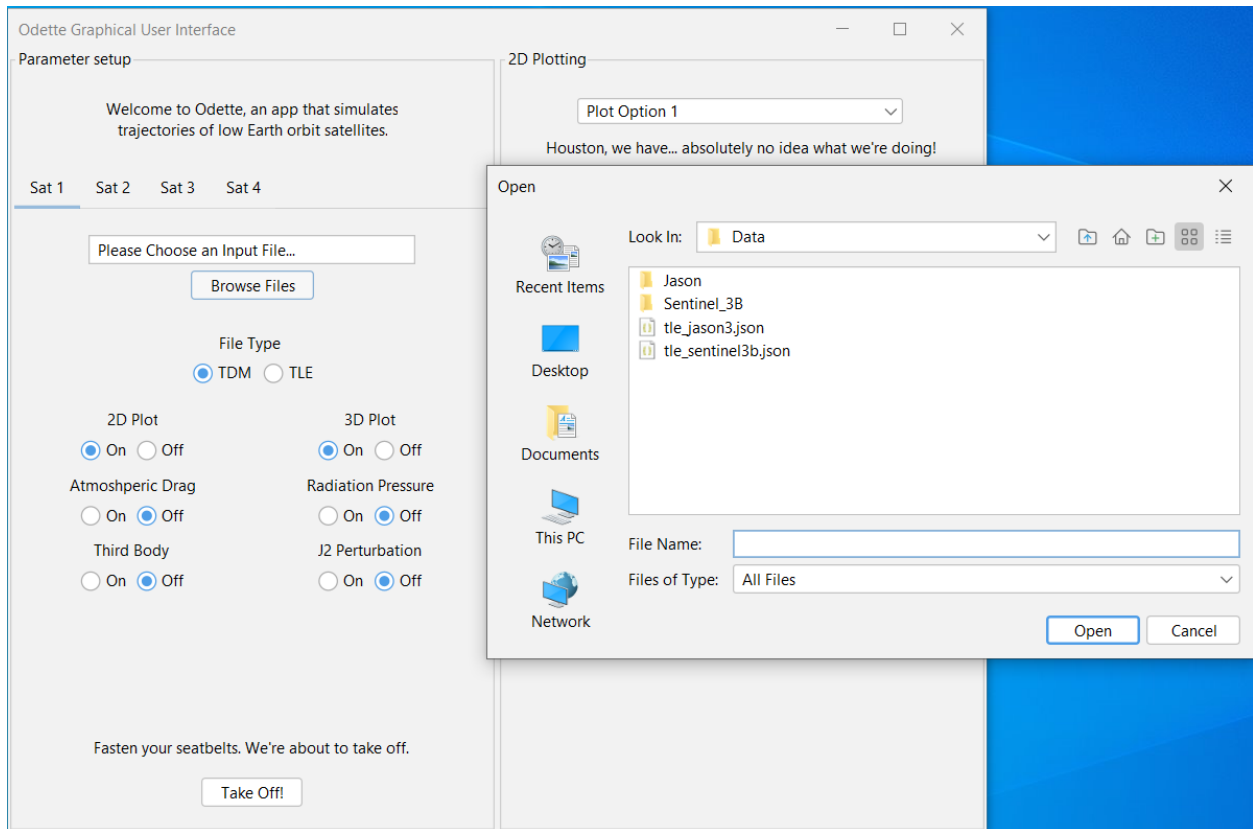


Figure 17: Choosing the simulation input files.

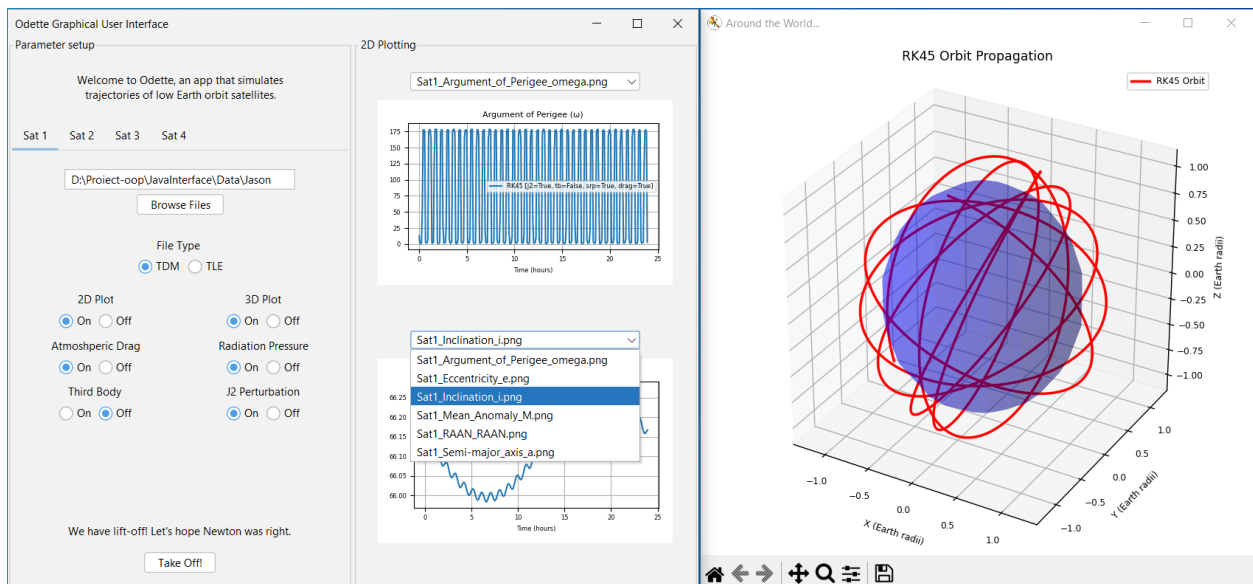


Figure 18: The graphical interface upon successfully running an orbit simulation.

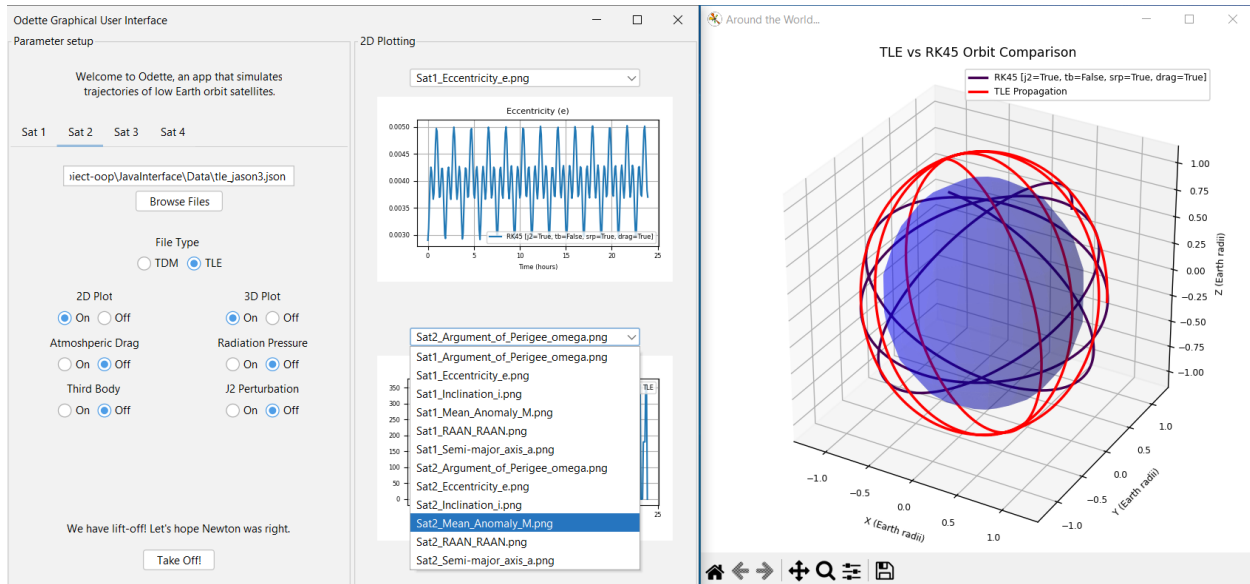


Figure 19: Simulating TDM and TLE orbits at the same time.

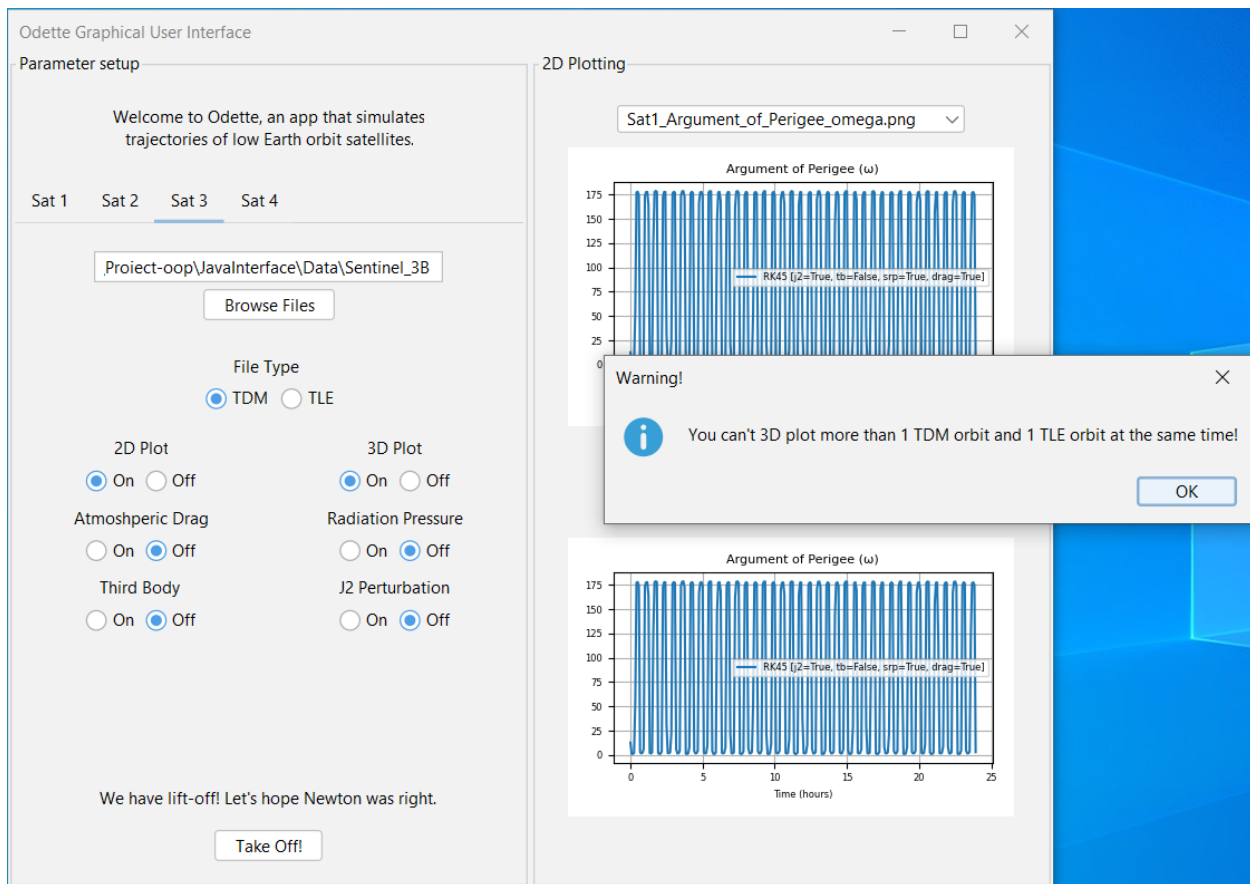


Figure 20: Error encountered when maximum number of satellites for 3D plotting is crossed.

3.5.3 Implementation

The Java code for the Odette graphical interface consists in 9 classes, together with one interface that is implemented by some of the classes. The classes vary in the complexity of the code they contain, as well as the tasks that they perform. Below, we provide a list of the Java classes, with brief explanations regarding their functionality.

- **GuiMain**

This is the driver class for the Odette graphical interface. For the most part, **GuiMain** contains standard code used for starting up a graphical interface. One important feature is that here we change the Look and Feel of the interface for a more modern aspect of our application. The LaF we have used is the **FlatLightLaf**, imported from the **flatlaf** package, as detailed in the section regarding dependencies.

- **BackgroundGrid**

The **BackgroundGrid** class is again straightforward, representing a **Panel** that uses the grid layout in order to have good control over the positions of the main areas of our interface: the left panel and the right panel, which are instantiated inside the **BackgroundGrid**. One important aspect is that this class enables the communication between the left and right panels, such that when the start button is pressed on the left panel, the resulting plots are displayed on the right panel. This is achieved by passing a reference to the **BackgroundGrid** class in the **LeftPanel** class. In this way, the left panel can call the **updateComboBoxes()** method of the **BackgroundGrid**, which then sends the command further to the right panel.

Methods:

- **public void updateComboBoxes()** - This method is called by the left panel so that the combo boxes on the right panel are updated when new plots are generated by pressing the Take Off button on the left panel. The **BackgroundGrid** class mediates this process.

- **LeftPanel**

The **LeftPanel** is the most important part of the graphical interface. It contains a **ParameterPanel** for each satellite, through which we individually set the parameters for the orbit simulation. The **LeftPanel** contains the Take Off button, which starts the simulation when it is pressed, by calling the **runForrest()** method, detailed below.

Methods:

- **private ArrayList<HashMap<String, Boolean>> buildPerturbation(ParameterPanel pp)** - This method is responsible for collecting the perturbation parameters set on each **ParameterPanel** and putting them together in an **ArrayList** which contains one **HashMap** element, with keys corresponding to the perturbation names, and values given by the true/false status of the perturbations.
- **private String[] getTleLines(String filePath)** - Method used for reading a JSON file with TLE data and returning the TLE information as a list of two strings.
- **private JSONObject buildPlotCommand(int dimension, String plotType, int i)** - Method responsible for creating the **JSONObject** that is sent as input to the Python functions for plotting. It takes as input the dimension of the required plot (2D/3D), the type of the plot, and the index *i* corresponding to the **ParameterPanel** from which we take the simulation parameters.
- **private void runForrest()** - Method run when the Take Off button is pressed. It goes through all the **ParameterPanels** to collect the simulation parameters and proceeds to call the Python functions responsible for plotting. Here, the connection between Java and Python is realized through the JEP library (Java Embedded Python). More specifically, we use the **SharedInterpreter** class from JEP to call the Python plotting functions.

- **ParameterPanel**

The **ParameterPanel** contains components used to set simulation parameters for each satellite: choice buttons and a text field for the file path, together with a file browsing button. The **ParameterPanel** is based on the **MultipleChoicePanel** class.

Methods:

- **public boolean[] getChoices()** - Returns a list of the boolean values corresponding to the chosen simulation parameters: types of chosen plots and the four orbit perturbations.
- **public boolean[] getFileType()** - Returns a list with a single boolean value corresponding to the chosen input file type. True corresponds to TDM files, while False corresponds to TLE files.
- **public String getFilePath()** - Returns a String representing the absolute path to the chosen input file.

- **MultipleChoicePanel**

The **MultipleChoicePanel** groups together **TwoChoicePanel** objects used for setting each of the individual simulation parameters.

Methods:

- **public boolean[] getChoices()** - Returns a list of the boolean values corresponding to the chosen simulation parameters: types of chosen plots and the four orbit perturbations.

- **TwoChoicePanel**

The **TwoChoicePanel** is used to set each simulation parameter; it contains the name of the parameter and a set of on/off buttons.

Methods:

- **public boolean[] getChoices()** - Returns a list of a single boolean value corresponding to the on/off state of the chosen simulation parameter.

- **OnOffButtons**

Create a pair of on/off buttons to set each simulation parameter.

Methods:

- **public boolean[] getChoices()** - Returns a list of a single boolean value corresponding to the on/off state of the chosen simulation parameter.

- **RightPanel**

This class represents the right window of the interface, responsible for displaying the generated plots for the orbital elements of the satellite orbits. The Right Panel contains two combo boxes that list all the generated plots from which the user can choose. In this way, orbital elements of different satellite orbits can be compared, for example.

Methods:

- **public void updateComboBoxOptions()** - Method responsible for updating the combo boxes with the newly generated plots each time a new simulation is run. The plots are saved in a folder, and then opened into the interface. This method is run from the LeftPanel when pressing the Take Off button, using the BackgroundGrid as an intermediary, since both LeftPanel and RightPanel are instantiated inside the BackgroundGrid.
- **private void showSelectedPlot()** - Method that displays selected plot when an option is chosen in one of the combo boxes.

- **PathRegistry**

Singleton class used for storing various system paths used by the other classes for file processing.

Methods:

- `public static PathRegistry getInstance()` - Returns the single instance of the Singleton class.
- `public String getProjectDir()` - Returns a String representing the absolute path of the Java project directory.
- `public String getPlotDir()` - Returns a String representing the absolute path of the directory where generated plots are saved to be later displayed on the right panel.
- `public String getPythonDir()` - Returns a String representing the absolute path of the directory of the Python module whose functions we call for computing orbits.
- `public String getDataDir()` - Returns a String representing the absolute path of the directory where simulation input files are stored.

- **Choice**

Interface defined for implementing the **Composite Design Pattern**.

Methods:

- `public boolean[] getChoices()` - Abstract method implemented by a number of classes: `OnOffButtons`, `TwoChoicePanel`, `MultipleChoicePanel`, `ParameterPanel`. In this way, we achieve the structure of leaf and container elements characteristic to the Composite pattern.

3.6 Output Formats and Result Interpretation

All interfaces allow exporting results to common formats:

- **CSV:** Position and velocity vectors over time.
- **JSON:** Full state including propagation metadata.
- **Images:** Plots of orbital elements, trajectories, and error comparisons.

Users are encouraged to interpret propagation outputs by comparing across models and evaluating consistency over time. The RK45 method is more accurate but sensitive to step size and perturbations, whereas SGP4 is analytical and faster but less accurate for perturbed scenarios.

3.7 Tutorials and Documentation Access

Tutorial videos are available covering:

- Setting up the Odette system.
- Running TLE simulations in Python.
- Fitting RA/Dec observations using TDM.
- Navigating the Java GUI for visualization.

Documentation is generated using Doxygen and is available in:

- **HTML:** Navigable, with hyperlinks between classes and functions.
- **LaTeX/PDF:** Printable reference manual.
- **Inline docstrings:** Within the code.